AI & Data Science Track – Round 4

Advanced Food Image Generation System

Text-to-Image Generation for Restaurant & Food Delivery Platforms

Project Submission – Planning, Literature Review & Requirements

Team Members

Omar AbdelNasser (Team Leader) · Mohamed Tarek · Marcelino Maged · Verina Antonious · Momen Mohamed

Section 1: Project Planning & Management

1.1 Project Proposal

Overview

This project develops a food-focused text-to-image generation system leveraging a fine-tuned Stable Diffusion model adapted to the food domain using Low-Rank Adaptation (LoRA). The system is designed to serve restaurant owners and food delivery platforms (such as Talabat or Uber Eats) by automatically generating high-quality, photorealistic food images from short textual menu descriptions — eliminating the need for expensive professional food photography.

Approach

Rather than training a generative model from scratch, the project employs transfer learning by fine-tuning a pretrained Stable Diffusion v1.5 model on a curated food subset of the COCO dataset. LoRA adapters are applied specifically to the cross-attention layers of the U-Net, allowing the model to specialize in food image generation while preserving the broad visual knowledge of the base model. CLIP serves as the frozen text encoder, converting menu-style text prompts into semantic embedding vectors that condition the diffusion process. This approach is consistent with the project requirement to use transfer learning where pretrained models are available, and directly addresses the attention mechanism requirement by targeting the cross-attention layers within the U-Net denoiser.

Objectives

- Curate and preprocess a high-quality food image dataset by filtering the COCO dataset (jxie/coco_captions) using CLIP-based relevance scoring to retain only genuinely food-relevant image-caption pairs across 20 food categories.
- Fine-tune a pretrained Stable Diffusion v1.5 U-Net using LoRA adapters targeting the cross-attention layers (to_q, to_k, to_v), enabling food-domain specialization with minimal compute.
- Leverage the frozen CLIP text encoder for semantically rich text conditioning, ensuring generated images align closely with input food descriptions.
- Implement classifier-free guidance during inference to improve prompt adherence and photorealism of generated food images.
- Deploy the model through a FastAPI web interface enabling real-time image generation from restaurant menu text prompts.
- Implement MLOps best practices (MLflow, TensorBoard, Docker) for experiment tracking, model versioning, and reproducible deployment.

Scope

- In scope: COCO food subset curation, CLIP relevance filtering, LoRA fine-tuning of SD v1.5, CLIP text encoding, classifier-free guidance inference, FastAPI deployment, MLOps pipeline, FID/IS evaluation metrics.

- Out of scope: Mobile app development, multi-language support, video generation, training a full diffusion model from scratch, real-time photorealistic rendering at commercial scale.

1.2 Project Plan

Timeline & Milestones

The project follows the official course deadlines across six submission phases:

Phase	Deliverables	Deadline
Project Planning & Management	Project proposal, Gantt chart, task assignment, risk assessment, KPIs	Feb 20, 2026
Literature Review	Review of diffusion models, LoRA, CLIP, attention mechanisms, food datasets	Apr 20, 2026
Requirements Gathering	Stakeholder analysis, user stories, functional & non-functional requirements	Apr 20, 2026
System Analysis & Design	Architecture diagrams, data pipeline design, model design, API design	May 1, 2026
Implementation (Source Code & Execution)	Working notebook: data pipeline, LoRA fine-tuning, inference, FastAPI, MLOps	Jul 10, 2026
Final Presentation, Testing & Reports	Final report, test results (FID/IS), live demo, presentation slides	Jul 17, 2026

Internal Milestone Breakdown

Milestone	Description	Target Week
M1 – Data Pipeline	COCO food subset streaming, CLIP relevance filtering, EDA, preprocessing, embedding precomputation	Weeks 1–2
M2 – LoRA Fine-tuning	SD v1.5 LoRA setup, training loop with gradient accumulation, checkpoint saving	Weeks 3–5
M3 – Inference Pipeline	DDIM sampler, classifier-free guidance, prompt engineering, quality evaluation	Week 6
M4 – MLOps	MLflow experiment tracking, TensorBoard visualization, Docker containerization	Week 7
M5 – Deployment & Demo	FastAPI endpoint, web UI, FID/IS scoring, final report, presentation	Week 8

Resource Allocation

Resource	Purpose	Cost
Kaggle Notebooks (T4 GPU)	LoRA fine-tuning and inference (30 hrs/week free)	Free
Hugging Face Hub	Dataset access (jxie/coco_captions) and model weights (SD v1.5)	Free
Python + PyTorch + Diffusers	Core ML and diffusion framework	Free / Open source
PEFT library	LoRA adapter implementation	Free / Open source
MLflow	Experiment tracking and model versioning	Free / Open source
Docker	Containerization for reproducible deployment	Free
FastAPI	REST API for model serving and demo	Free / Open source

1.3 Task Assignment & Roles

All five team members collaborate across all project phases. Responsibilities are distributed to ensure every member contributes to both the technical implementation and documentation, with specialization areas noted below.

Team Member	Role	Primary Responsibilities
Omar AbdelNasser (Team Lead)	Team Lead & ML Engineer	Overall project coordination and technical direction. CLIP text encoder integration and embedding pipeline. LoRA training loop implementation and hyperparameter tuning. Classifier-free guidance inference pipeline. Code review and integration across all modules.
Mohamed Tarek	Data & ML Engineer	COCO dataset streaming, food category filtering, and CLIP relevance scoring pipeline. Data preprocessing and augmentation. Collaboration on LoRA fine-tuning experiments and evaluation metrics (FID/IS). Contribution to system design documentation.
Marcelino Maged	ML & MLOps Engineer	LoRA adapter configuration and attention mechanism targeting. MLflow experiment tracking setup and TensorBoard visualization. Model versioning and checkpoint management. Collaboration on data pipeline and preprocessing decisions.
Verina Antonious	ML & Backend Engineer	Diffusion model inference pipeline and DDIM sampler implementation. FastAPI endpoint development for text-to-image serving. Collaboration on LoRA training and evaluation. Web UI implementation for the live demo.
Momen Mohamed	ML & Backend & Deployment Engineer	Docker containerization and deployment configuration. FastAPI integration and API testing. Collaboration on MLOps pipeline setup. Final report compilation, presentation preparation, and live demo coordination.

1.4 Risk Assessment & Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Kaggle GPU session timeout mid-training	High	High	Checkpoint saved every epoch to /kaggle/working; reload cell restores full state in seconds. LoRA weights are tiny (<50MB) so saving is fast.
LoRA fine-tuning produces low-quality images	Medium	High	Increase LoRA rank (r=16), use gradient accumulation, add negative prompts and higher guidance scale during inference. Fall back to more epochs if needed.
Base SD model too large for available VRAM	Medium	High	Load all components in float16; use LoRA instead of full fine-tuning; batch size of 1 with gradient accumulation of 4.
COCO food subset too small or noisy	Low	Medium	CLIP relevance scoring filters out non-food images; category balancing ensures diverse training signal across 20 food types.
Team member unavailability near deadlines	Low	Medium	All code in shared Kaggle notebook with clear cell structure; weekly syncs to track progress; overlapping responsibilities prevent single points of failure.
Dependency/environment conflicts	Low	Low	All packages pinned in requirements.txt; Docker container ensures identical environment across machines.

1.5 Key Performance Indicators (KPIs)

KPI	Metric	Target
Image Quality	Fréchet Inception Distance (FID)	FID < 80 on food validation set (achievable with LoRA fine-tuning)
Text-Image Alignment	Inception Score (IS)	IS > 4.0
Prompt Adherence	CLIP Score (text-image cosine similarity)	> 0.25 on held-out food prompts
System Responsiveness	API response time per image	< 15 seconds per image on T4 GPU
Model Reproducibility	MLflow tracked runs	100% of training runs logged with params, metrics, and artifacts
Dataset Quality	CLIP relevance score after filtering	Mean CLIP food score > 0.75 across retained samples
System Uptime	FastAPI availability during demo	> 95% uptime during demonstration session

Section 2: Literature Review

2.1 Introduction

Text-to-image generation has undergone a dramatic transformation over the past five years, progressing from unstable GAN-based methods to highly capable diffusion models that produce photorealistic images. This review examines the foundational research underpinning this project, with particular focus on diffusion models, transfer learning via LoRA, CLIP-based text conditioning, and cross-attention mechanisms — the four pillars of the adopted approach.

2.2 Generative Adversarial Networks — Historical Context

The first deep learning approaches to text-to-image synthesis used Generative Adversarial Networks (GANs). Reed et al. (2016) were among the first to condition image generation on text embeddings, producing plausible bird and flower images. StackGAN (Zhang et al., 2017) extended this with a two-stage architecture producing higher resolution outputs. AttnGAN (Xu et al., 2018) introduced word-level attention, enabling the generator to focus on specific words when rendering specific image regions. While GAN-based methods were foundational, they suffer from training instability (mode collapse), sensitivity to hyperparameters, and difficulty scaling to complex, diverse domains such as general food imagery. This motivated the shift toward diffusion models as the primary generative framework in this project.

2.3 Diffusion Models

Denoising Diffusion Probabilistic Models (DDPM), introduced by Ho et al. (2020), define a forward process that gradually adds Gaussian noise to an image over T timesteps, and a learned reverse process — implemented as a U-Net — that iteratively denoises. Unlike GANs, diffusion models have a stable variational lower-bound training objective and naturally produce diverse, high-fidelity outputs. Nichol and Dhariwal (2021) demonstrated that diffusion models surpass GANs on standard image quality benchmarks. Song et al. (2021) introduced DDIM sampling, reducing inference steps from 1000 to 50 without quality degradation — a technique employed in this project's inference pipeline. Rombach et al. (2022) introduced Latent Diffusion Models (LDM), operating the diffusion process in a compressed VAE latent space rather than pixel space, dramatically reducing compute requirements. Stable Diffusion, the base model used in this project, is an implementation of LDM trained on LAION-5B.

2.4 Transfer Learning and Low-Rank Adaptation (LoRA)

Fine-tuning large pretrained models on domain-specific data is a well-established transfer learning paradigm. However, full fine-tuning of a 860M-parameter model such as Stable Diffusion's U-Net is computationally prohibitive on consumer-grade hardware. Hu et al. (2021) introduced Low-Rank Adaptation (LoRA), which freezes the pretrained model weights and injects trainable low-rank decomposition matrices into the attention layers. Specifically, for a weight matrix W, LoRA learns two matrices A and B where rank(AB) << rank(W), such that the effective weight update is W + AB. This reduces trainable parameters by orders of magnitude — in this project, from 860M to approximately 3M — while achieving comparable fine-tuning

performance. LoRA has been widely adopted for Stable Diffusion customization, with the Kohya-SS and diffusers/PEFT implementations becoming standard tools. This project applies LoRA to the cross-attention projection layers (to_q, to_k, to_v, to_out) of the U-Net, which are the layers most responsible for text-image alignment.

2.5 CLIP as a Text Encoder

Contrastive Language-Image Pretraining (CLIP), introduced by Radford et al. (2021) at OpenAI, was trained on 400 million image-text pairs to align visual and textual representations in a shared embedding space. Unlike BERT (which processes text only), CLIP's text encoder inherently understands the relationship between language and visual concepts, producing text embeddings that are semantically compatible with image features. This makes CLIP a far superior conditioning signal for image generation than a purely text-trained encoder. Stable Diffusion uses the CLIP ViT-L/14 text encoder as its conditioning backbone — the same encoder is leveraged in this project (frozen throughout training) to encode food menu descriptions into 768-dimensional conditioning vectors.

2.6 Cross-Attention Mechanisms in Diffusion U-Nets

The cross-attention mechanism allows the denoising U-Net to selectively attend to different parts of the text conditioning at each spatial location of the image being generated. In the U-Net architecture used by Stable Diffusion, cross-attention layers are interleaved with the residual convolutional blocks at multiple resolution levels. At each cross-attention layer, spatial image features form the query, while the CLIP text embedding forms the keys and values. This enables the model to ask, at each image region, which words are most relevant to what should appear there. Vaswani et al. (2017) introduced the scaled dot-product attention that underlies this mechanism. The LoRA adapters in this project target precisely these cross-attention layers, reinforcing the food-domain alignment between visual features and text descriptions.

2.7 Classifier-Free Guidance

Classifier-free guidance (Ho and Salimans, 2022) is an inference-time technique that improves text-image alignment without requiring a separate classifier model. During training, the conditioning signal is randomly dropped (replaced with a null embedding) with some probability, teaching the model to generate both conditional and unconditional outputs. At inference time, the predicted noise is extrapolated in the direction of the conditional prediction away from the unconditional prediction: pred = uncond + scale * (cond - uncond). Higher guidance scales push the output more strongly toward the text prompt at the cost of some diversity. This project employs classifier-free guidance with a scale of 7.5–9.0 during inference, which is critical for generating food images that visually match the input menu description.

2.8 Food Image Datasets and Related Work

The COCO dataset (Lin et al., 2014) contains 123,000 images across 80 object categories with 5 human-annotated captions per image. A significant subset covers food items including pizza, sandwiches, cakes, hot dogs, broccoli, and fruits. The jxie/coco_captions Hugging Face version

provides 567k training rows with clean image-caption pairs. After CLIP-based relevance filtering and category balancing, this project retains approximately 10,570 high-quality food image-caption pairs across 20 categories as the fine-tuning dataset. Domain-specific food image generation has direct commercial applications in food delivery platforms, where professional food photography is expensive and inaccessible to small restaurant owners.

2.9 Key References

1. Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. NeurIPS 2020.
2. Hu, E., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
3. Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). ICML 2021.
4. Rombach, R., et al. (2022). High-Resolution Image Synthesis with Latent Diffusion Models. CVPR 2022.
5. Song, J., et al. (2021). Denoising Diffusion Implicit Models (DDIM). ICLR 2021.
6. Nichol, A., & Dhariwal, P. (2021). Improved Denoising Diffusion Probabilistic Models. ICML 2021.
7. Ho, J., & Salimans, T. (2022). Classifier-Free Diffusion Guidance. NeurIPS Workshop 2021.
8. Xu, T., et al. (2018). AttnGAN: Fine-Grained Text to Image Generation with Attentional GANs. CVPR 2018.
9. Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.
10. Lin, T.Y., et al. (2014). Microsoft COCO: Common Objects in Context. ECCV 2014.

2.10 Suggested Improvements for Future Work

- Fine-tune on a larger food-specific dataset such as Food-101 or Recipe1M+ to improve domain coverage and generalization beyond COCO food categories.
- Increase LoRA rank progressively (r=32 or r=64) with more GPU budget to capture finer food texture details.
- Explore DreamBooth fine-tuning for ultra-specific food item personalization (e.g., a restaurant's signature dish from 5 reference photos).
- Extend to image editing via inpainting, allowing restaurant owners to modify specific elements of existing food photos using text instructions.
- Implement latent consistency models (LCM-LoRA) for 4-step inference, reducing generation time to under 1 second per image.

Section 3: Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Type	Needs & Expectations
Restaurant Owners / Small Businesses	Primary User	Generate appetizing food images from menu text without hiring photographers; fast turnaround; easy-to-use interface requiring no technical knowledge
Food Delivery Platforms (Talabat, Uber Eats, etc.)	Primary Beneficiary	Higher-quality menu listings across all restaurants; automated image pipeline for onboarding new vendors; improved customer conversion rates
End Customers (App Users)	Indirect Beneficiary	Accurate, appetizing food visuals that match what they order; builds trust in the platform
Course Lecturer / Academic Evaluator	Internal Stakeholder	Demonstrated understanding of diffusion models, transfer learning, attention mechanisms, and MLOps; working implementation with documented results
Development Team (5 members)	Internal Stakeholder	Clear task division, manageable compute requirements, reproducible experiments, well-structured codebase

3.2 User Stories & Use Cases

User Story 1 – Restaurant Owner

As a restaurant owner, I want to type a menu item description such as 'grilled salmon with lemon and herbs on a white plate' and receive a realistic, appetizing food image within seconds, so that I can upload it to my delivery app listing without needing a professional photographer.

User Story 2 – Platform Operator

As a food delivery platform operator, I want restaurants without photos to be able to auto-generate images via a single API call, so that every menu item has a visual representation and my platform's listing quality improves at scale without manual intervention.

User Story 3 – Developer / Integrator

As a developer integrating the system into an existing restaurant onboarding workflow, I want a well-documented REST API endpoint that accepts a text string and returns a PNG image, so that I can automate image generation for newly onboarded restaurants.

Use Case: Generate Food Image from Text

Field	Description
Use Case ID	UC-01
Name	Generate food image from text description
Actor	Restaurant owner or API client
Precondition	System is running; FastAPI service is active; LoRA fine-tuned model is loaded
Main Flow	1. User submits text prompt via web UI or API POST request. 2. CLIP text encoder converts prompt to a 768-dim embedding vector. 3. Classifier-free guidance generates null embedding for unconditional branch. 4. DDIM sampler runs 50 reverse diffusion steps in VAE latent space, guided by text embedding. 5. VAE decoder reconstructs the image from the denoised latent. 6. Generated PNG image is returned to the user.
Alternate Flow	If text prompt is empty or exceeds 77 CLIP tokens, system returns HTTP 400 with guidance message.
Postcondition	A photorealistic food image is returned matching the description.
Success Metric	Generated image is visually coherent with food content matching the prompt; response delivered within 15 seconds.

3.3 Functional Requirements

ID	Requirement	Priority
FR-01	System shall load and filter the COCO food category subset from jxie/coco_captions via the Hugging Face datasets library using keyword matching and CLIP relevance scoring.	Must Have
FR-02	System shall preprocess images to 512×512 pixels with normalization to [-1,1] and apply data augmentation (horizontal flip, color jitter) during training.	Must Have
FR-03	System shall use a frozen CLIP text encoder (from Stable Diffusion v1.5) to encode text prompts into 768-dimensional conditioning vectors.	Must Have
FR-04	System shall apply LoRA adapters (rank=16) to the cross-attention layers (to_q, to_k, to_v, to_out) of the SD v1.5 U-Net and fine-tune on the food dataset.	Must Have
FR-05	System shall use a frozen VAE encoder/decoder to operate the diffusion process in latent space, reducing memory and compute requirements.	Must Have
FR-06	System shall implement DDIM sampling with classifier-free guidance (scale 7.5–9.0) for inference, generating images in 50 steps.	Must Have
FR-07	System shall expose a FastAPI endpoint (POST /generate) accepting a JSON body with a 'prompt' field and returning a PNG image.	Must Have
FR-08	System shall compute FID and Inception Score on a held-out food validation set to quantitatively evaluate generation quality.	Should Have
FR-09	System shall log all training parameters, loss curves, and evaluation metrics to MLflow for full experiment tracking.	Must Have

ID	Requirement	Priority
FR-10	System shall be packaged in a Docker container with Dockerfile and requirements.txt for reproducible deployment.	Must Have
FR-11	System shall support negative prompt input at inference time to suppress unwanted styles (e.g., cartoon, blurry).	Should Have
FR-12	System shall provide a simple web UI (Gradio or HTML form) for non-technical users to enter prompts and view generated images.	Should Have

3.4 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Single image generation shall complete within 15 seconds on a T4 GPU using 50 DDIM steps.
NFR-02	Performance	The system shall support at least 5 concurrent API requests without crashing or significant quality degradation.
NFR-03	Reliability	The FastAPI service shall achieve at least 95% uptime during the demonstration session.
NFR-04	Scalability	The Docker container shall be deployable on any machine with Docker installed and a CUDA-compatible GPU with at least 8GB VRAM.
NFR-05	Usability	A non-technical restaurant owner with no ML background shall be able to generate an image within 2 interactions with the web UI.
NFR-06	Reproducibility	All training runs shall be reproducible by setting a fixed random seed and logging all hyperparameters (LoRA rank, learning rate, epochs, guidance scale) to MLflow.
NFR-07	Maintainability	All code shall be organized into clearly named modules (data/, model/, train/, api/) with docstrings and inline comments.
NFR-08	Security	The API shall validate and sanitize all input prompts; maximum input length shall be enforced at 77 CLIP tokens.
NFR-09	Portability	The project shall run on Python 3.10+ with all dependencies declared in requirements.txt with pinned versions.
NFR-10	Ethical	The model shall be fine-tuned only on the COCO dataset under its CC BY 4.0 license; the base Stable Diffusion model shall be used in compliance with its CreativeML Open RAIL-M license.